



Département des Technologies de l'information et de la
communication (TIC)
Filière Informatique et systèmes de communication
Orientation Sécurité informatique

Rapport de recherche

Selfie vidéo

Étudiant

Enseignant responsable

Année académique

Dylan Oliveira Ramos

Prof. Jean-Marc Bost

2025-2026

Yverdon-les-Bains, le 08.06.2026

Table des matières

1	Introduction	4
1.1	Facebook	4
1.2	Parship	6

Table des figures

Fig. 1 E-mail de suspension du compte lors de l'utilisation d'un alias.	4
Fig. 2 Personne fictive générée.	5
Fig. 3 Échec de la vérification d'identité.	5
Fig. 4 Échec de la vérification d'identité en utilisant une caméra virtuelle.	6
Fig. 5 Échec de la vérification d'identité avec une caméra réelle.	7
Fig. 6 Modification du module v4l2loopback.	8
Fig. 7 Caméra créée avec le module original.	9
Fig. 8 Caméra créée avec le module modifié.	9

1 Introduction

La plupart des entreprises développant des systèmes de vérification d'identité en ligne ne divulguent pas la manière dont ils vérifient les identités. Ainsi, il est très difficile voire impossible d'identifier une méthode de contournement qui fonctionne à tous les coups. Ce document présente les différents tests effectués pour essayer de comprendre et de contourner les systèmes de vérification par selfie vidéo.

1.1 Facebook

Lors de la création d'un compte, Facebook demande de prendre un selfie vidéo en tournant la tête dans différentes directions. Les directions étant aléatoires, j'ai dû me filmer sur le moment puis demander à l'IA de remplacer mon visage par celui d'une personne fictive.

Un point important à prendre en compte est que Facebook bloque automatiquement les comptes créés avec un alias comme adresse e-mail. J'ai donc dû créer une adresse Gmail pour chacune de mes tentatives.

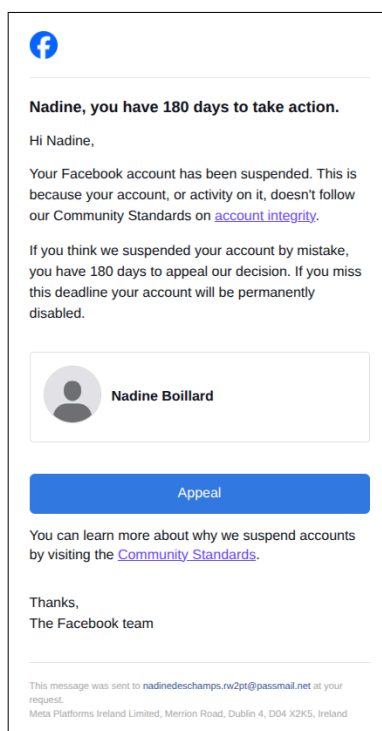


Fig. 1. – E-mail de suspension du compte lors de l'utilisation d'un alias.

Une fois le compte créé, la vérification d'identité est demandée. Pour pouvoir essayer de la contourner, je dois d'abord savoir quels sont les mouvements demandés, pour cela je diffuse un écran noir sur la caméra virtuelle car une caméra doit être détectée pour que la vérification commence. La vidéo [facebook-1.webm](#) démontre le type de mouvements demandés par Facebook.

Une fois les mouvements connus, je me suis filmé en train de les faire, puis j'ai demandé à l'IA de remplacer mon visage par celui d'une personne fictive, le tout en gardant la page de vérification ouverte.

Génération de la personne :

- Modèle : nano-banana-2

- Prompt : A headshot portrait of a young woman in her early 20s, calm and neutral facial expression, looking directly forward at the viewer, sharp focus on the face, passport-style portrait photography.
- Format d'image : auto
- Résolution : 1K



Fig. 2. – Personne fictive générée.

Génération de la vidéo :

- Modèle : happyhorse/video-edit
- Prompt : Replace the man on the video by the woman on the image.
- Resolution : 720p

Résultat : [facebook-2.mp4](#).

Enfin, j'ai diffusé la vidéo sur la caméra virtuelle et je suis retourné sur la page de vérification, le résultat est le suivant : [facebook-3.webm](#). Et après quelques heures d'attente, j'ai reçu un e-mail de Facebook m'informant que ma vérification d'identité avait échoué.

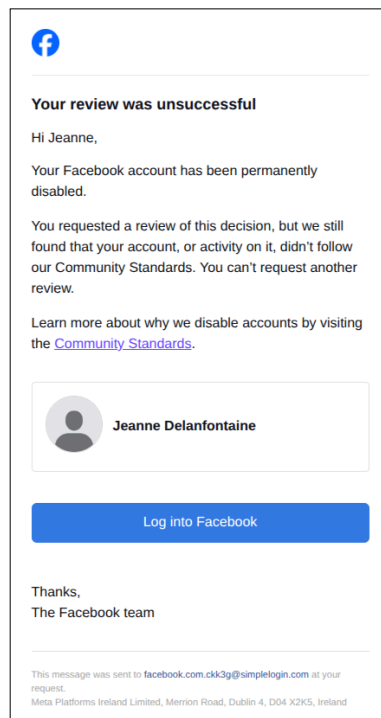


Fig. 3. – Échec de la vérification d'identité.

Après plusieurs tentatives similaires, j'ai toujours reçu le même résultat. Je suis maintenant dans l'incapacité d'en faire d'autres car je ne peux plus créer d'adresses Gmail, mon numéro de téléphone ayant été utilisé trop de fois.

1.2 Parship

Comme pour Facebook, Parship demande de prendre un selfie vidéo lors de la création d'un compte. Cependant, aucun mouvement n'est demandé, il faut simplement centrer son visage dans un oval puis se rapprocher de la caméra. Je n'ai donc pas eu besoin de me filmer préalablement, j'ai directement demandé à l'IA de générer une vidéo d'une personne fictive en train de se filmer :

- Modèle : kling-3.0/video
- Prompt : The woman in the image looks into the camera for a while, her body is not moving. Then the camera slowly zooms to her face and the woman continues to look at the camera. Then the camera zoom more and the woman is still looking at the camera. Then the camera slowly goes back.
- Durée : 15s
- Format d'image : 16:9

Résultat : [parship-1.mp4](#).

J'ai ensuite diffusé la vidéo sur la caméra virtuelle et je suis retourné sur la page de vérification, le résultat est le suivant : [parship-2.mp4](#). La vérification a échoué, mais contrairement à Facebook, Parship nous dit pourquoi la vérification a échoué, comme nous pouvons le voir à la fin de la vidéo.



Fig. 4. – Échec de la vérification d'identité en utilisant une caméra virtuelle.

Nous pouvons en déduire que le système de vérification de Parship analyse également le type de caméra qui est utilisé. J'ai donc essayé de refaire la vérification, mais cette fois-ci avec une caméra réelle pour voir si le résultat était différent. Pour cela, j'ai affiché l'image de la personne fictive sur mon écran et j'ai filmé cette image avec une webcam.

Résultat : [parship-3.mp4](#).

Le résultat est bien différent, cela confirme donc que le système de vérification de Parship analyse le type de caméra utilisé et que la diffusion d'une vidéo sur une caméra virtuelle est détectée et bloquée.

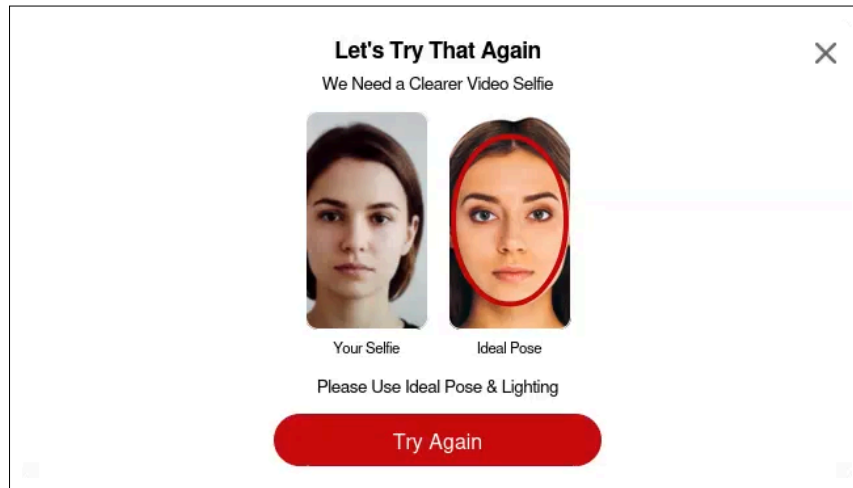


Fig. 5. – Échec de la vérification d'identité avec une caméra réelle.

Suite à cette découverte, j'ai cherché un moyen de faire croire au système de vérification que la caméra virtuelle était une caméra réelle. Pour cela, j'ai essayé de modifier le code source du module `v4l2loopback` utilisé pour créer la caméra virtuelle pour qu'il n'affiche pas les caractéristiques d'une caméra virtuelle mais celles d'une caméra réelle.

Étape 1 : cloner le dépôt GitHub du projet `v4l2loopback`.

```
1 git clone git@github.com:v4l2loopback/v4l2loopback.git
2 cd v4l2loopback
```

Étape 2 : ouvrir le fichier `v4l2loopback.c` et modifier les chaînes de caractères affichées par des `snprintf`.

```

dylan@udesktop:~/Documents/v4l2loopback$ git diff
diff --git a/v4l2loopback.c b/v4l2loopback.c
index 43ecbff..daf5934 100644
--- a/v4l2loopback.c
+++ b/v4l2loopback.c
@@ -74,7 +74,7 @@
     KERNEL_VERSION(V4L2LOOPBACK_VERSION_MAJOR, V4L2LOOPBACK_VERSION_MINOR, \
                     V4L2LOOPBACK_VERSION_BUGFIX)

-MODULE_DESCRIPTION("v4l2 loopback video device");
+MODULE_DESCRIPTION("AUKEY PC-W1: AUKEY PC-W1 video device");
MODULE_AUTHOR("Vasily Levin, "
              "Iohannes m zmoelnig <zmoelnig@iem.at>,"
              "Stefan Diewald,"
@@ -905,10 +905,10 @@ static int vidioc_querycap(struct file *file, void *fh,
                        int device_nr = v4l2loopback_get_vdev_nr(dev->vdev);
                        __u32 capabilities = V4L2_CAP_STREAMING | V4L2_CAP_READWRITE;

-                        strscpy(cap->driver, "v4l2 loopback", sizeof(cap->driver));
-                        snprintf(cap->card, sizeof(cap->card), "%s", dev->card_label);
+                        strscpy(cap->driver, "uvcvideo", sizeof(cap->driver));
+                        snprintf(cap->card, sizeof(cap->card), "AUKEY PC-W1: AUKEY PC-W1");
                        snprintf(cap->bus_info, sizeof(cap->bus_info),
-                                "platform:v4l2loopback-%03d", device_nr);
+                                "usb-0000:00:14.0-13");

                        if (dev->announce_all_caps) {
                                capabilities |= V4L2_CAP_VIDEO_CAPTURE | V4L2_CAP_VIDEO_OUTPUT;
@@ -2862,7 +2862,7 @@ static int v4l2_loopback_add(struct v4l2_loopback_config *conf, int *ret_nr)
                                "Dummy video device (0x%04X)", nr);
                        }
                        snprintf(dev->v4l2_dev.name, sizeof(dev->v4l2_dev.name),
-                                "v4l2loopback-%03d", nr);
+                                "AUKEY PC-W1: AUKEY PC-W1");

                        err = v4l2_device_register(NULL, &dev->v4l2_dev);
                        if (err)
dylan@udesktop:~/Documents/v4l2loopback$ |

```

Fig. 6. – Modification du module v4l2loopback.

Étape 3 : recompiler et réinstaller le module.

```

1 make
2 sudo make install
3 sudo depmod -a

```

Étape 4 : créer une nouvelle caméra virtuelle.

```

1 sudo modprobe v4l2loopback devices=1 video_nr=0 card_label="AUKEY PC-W1: AUKEY
   PC-W1" exclusive_caps=1

```



```
dylan@udesktop:~$ v4l2-ctl --device=/dev/video1 --info
Driver Info:
    Driver name      : v4l2 loopback
    Card type        : AIFRB Virtual Camera
    Bus info         : platform:v4l2loopback-001
    Driver version    : 7.0.0
    Capabilities     : 0x85200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
        Device Capabilities
    Device Caps      : 0x05200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
dylan@udesktop:~$ |
```

```
dylan@udesktop:~$ v4l2-ctl --device=/dev/video2 --info
Driver Info:
    Driver name      : uvcvideo
    Card type        : AUKEY PC-W1: AUKEY PC-W1
    Bus info         : usb-0000:00:14.0-13
    Driver version    : 7.0.0
    Capabilities     : 0x85200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
        Device Capabilities
    Device Caps      : 0x05200002
        Video Output
        Read/Write
        Streaming
        Extended Pix Format
dylan@udesktop:~$ |
```

Fig. 7. – Caméra créée avec le module original. Fig. 8. – Caméra créée avec le module modifié.

Cependant, malgré toutes ces modifications, j’obtiens le même résultat, Parship détecte toujours que la caméra utilisée est une caméra virtuelle.